

Detecting Machine-Generated Russian Texts

Mariia Bratchenko

Spring 2026

Abstract

This project addresses the task of detecting machine-generated Russian texts. We use the RuATD 2022 dataset and formulate the problem as binary classification between human-written and machine-generated texts. We compare classical TF-IDF-based models, character-level models, combined word-and-character TF-IDF models, and a transformer-based RuBERT-tiny2 model. The models are evaluated using accuracy, macro-precision, macro-recall, and macro-F1. The best result is achieved by Word+Char TF-IDF with Logistic Regression, which obtains macro-F1 of 0.7036. The project code is available at: <https://github.com/bratchenko479-dot/RuATD>.

1 Introduction

The rapid development of neural language models has made machine-generated text increasingly fluent and difficult to distinguish from human-written text. This creates practical risks in areas such as spam generation, fake reviews, misinformation, automated news generation, and large-scale production of low-quality online content. Therefore, automatic detection of generated text is an important NLP task.

In this project, we study machine-generated Russian text detection using the RuATD 2022 dataset. We formulate the task as binary classification: given a Russian text, the model should predict whether it was written by a human or generated by a machine.

The main goal of this project is to build a reproducible experimental pipeline and compare several approaches to the task. We evaluate simple lexical baselines, character-level TF-IDF models, combined word-and-character TF-IDF models, and a transformer-based model. The best model in our experiments is Word+Char TF-IDF with Logistic Regression.

The main contributions of this project are:

- a reproducible pipeline for RuATD binary classification;
- comparison of majority, word-level TF-IDF, character-level TF-IDF, combined TF-IDF, and RuBERT-tiny2 models;
- dataset analysis with class balance and text length statistics;
- error analysis of the best-performing model.

1.1 Team

Maria Bratchenko prepared the dataset preprocessing pipeline, trained the models, evaluated the results, performed error analysis, prepared the repository, and wrote the final report.

2 Related Work

Artificial text detection is the task of distinguishing human-written texts from automatically generated texts. This task became especially important with the development of neural text generation models, because generated texts can imitate human writing style and may be used for spam, fake reviews, misinformation, or automated content production.

The main dataset used in this project is RuATD 2022, a shared task on artificial text detection in Russian [Shamardina et al., 2022]. The shared task was organized as part of the Dialogue Evaluation initiative. The dataset contains texts from 14 sources: one human source and 13 text generation models. The task includes two settings: binary classification, where the goal is to detect whether a text is human-written or machine-generated, and multiclass classification, where the goal is to identify the author or generator of a text.

The organizers of RuATD provided count-based and BERT-based baselines [Shamardina et al., 2022]. This makes the dataset suitable for comparing both classical machine learning methods and transformer-based approaches. In this project, we also compare these two families of methods: TF-IDF with linear classifiers and RuBERT-tiny2.

Several participant systems in RuATD used stronger neural methods. Maloyan et al. [Maloyan et al., 2022] proposed an ensemble of different pretrained models based on an attention mechanism and achieved first place in the binary task with accuracy of 0.82995 on the private test set. This result is substantially higher than the results obtained in this project. However, our goal is not to reproduce a large ensemble system, but to build a compact and reproducible course project with several competitive baselines.

Classical text classification methods often use TF-IDF features and linear classifiers such as Logistic Regression or Linear SVM. These methods are simple, fast, and often strong baselines for text classification. Character-level TF-IDF features can be especially useful for generated text detection because they capture surface-level patterns such as punctuation, word endings, orthographic regularities, and stylistic artifacts.

Transformer-based models such as BERT [Devlin et al., 2019] use contextual representations and are widely used for text classification. For Russian NLP, RuBERT-style models are commonly used as strong pretrained models. In this project, we use RuBERT-tiny2 as a lightweight transformer baseline.

3 Model Description

We formulate the problem as binary text classification. Given a Russian text x , the model predicts a label $y \in \{0, 1\}$, where 0 corresponds to a human-written text and 1 corresponds to a machine-generated text.

We compare seven models.

3.1 Majority Baseline

The majority baseline always predicts the most frequent class in the training data. Since the dataset is nearly balanced, this model is expected to achieve approximately 50% accuracy. It serves as a lower-bound baseline.

3.2 Word TF-IDF + Logistic Regression

The first lexical baseline uses word-level TF-IDF features and a Logistic Regression classifier. We use word n-grams from 1 to 2 and a maximum vocabulary size of 50,000 features.

TF-IDF represents each text as a sparse vector based on term frequency and inverse document frequency. Logistic Regression then learns a linear decision boundary between human-written and machine-generated texts.

3.3 Word TF-IDF + Linear SVM

The second lexical baseline uses the same word-level TF-IDF representation, but replaces Logistic Regression with Linear SVM. Linear SVM is a common strong baseline for text classification tasks.

3.4 Character TF-IDF Models

We also use character-level TF-IDF features. Character-level features are useful because machine-generated texts may differ from human texts not only in vocabulary, but also in surface-level properties such as punctuation, word endings, and character sequences.

We train two character-level models:

- Char TF-IDF + Logistic Regression;
- Char TF-IDF + Linear SVM.

For character-level TF-IDF, we use character n-grams from 3 to 5 and a maximum vocabulary size of 100,000 features.

3.5 Word+Char TF-IDF + Logistic Regression

The strongest classical model combines word-level and character-level TF-IDF features. The final feature representation is a union of word n-grams and character n-grams. This representation is then passed to Logistic Regression.

This model is our best-performing approach. It combines lexical information from word n-grams with surface-level stylistic information from character n-grams.

3.6 RuBERT-tiny2

The transformer-based baseline is RuBERT-tiny2. It is fine-tuned for binary sequence classification. Due to computational limitations, RuBERT-tiny2 was trained on a 10,000-example subset of the training data for one epoch. Therefore, this model should be interpreted as a lightweight transformer baseline rather than a fully optimized transformer solution.

4 Dataset

We use the RuATD 2022 dataset for Russian artificial text detection [Shamardina et al., 2022]. In this project, we use the binary classification setup. Each example consists of a Russian text and a binary label: human-written or machine-generated.

The original labels in the dataset are:

- H: human-written text;
- M: machine-generated text.

We convert these labels into numerical labels:

- H $\rightarrow$ 0;
- M $\rightarrow$ 1.

The original test file does not contain gold labels. Therefore, in this project we use the validation split as the evaluation set. The dataset is nearly perfectly balanced, as shown in Table 1.

Split	Human	Machine	Total
Train	64,535	64,531	129,066
Evaluation	10,756	10,755	21,511

Table 1: Class distribution in the RuATD binary classification data used in this project.

We also analyze text length statistics. Machine-generated texts are slightly longer on average, but the difference is small. The statistics are shown in Table 2.

Class	Avg. words	Median words	Avg. chars	Median chars
Human	30.07	13.00	221.47	95.00
Machine	31.85	14.00	236.91	99.00

Table 2: Text length statistics for human-written and machine-generated texts.

Figure 1 shows the class distribution in the training set. The classes are almost exactly balanced.

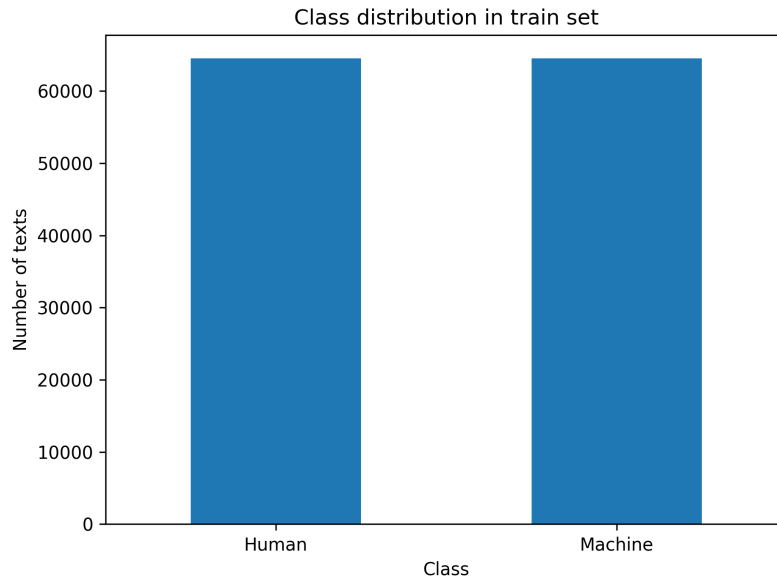


Figure 1: Class distribution in the training set.

Figure 2 shows the distribution of text lengths.

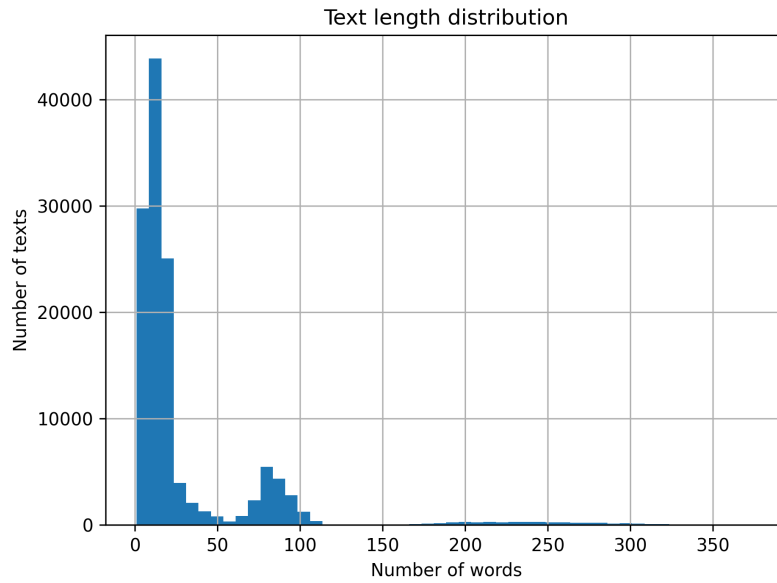


Figure 2: Text length distribution in the training set.

5 Experiments

5.1 Metrics

We evaluate all models using accuracy, macro-precision, macro-recall, and macro-F1.

Accuracy is defined as:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}. \quad (1)$$

Precision is the fraction of correctly predicted positive examples among all examples predicted as positive:

$$Precision = \frac{TP}{TP + FP}. \quad (2)$$

Recall is the fraction of correctly predicted positive examples among all true positive examples:

$$Recall = \frac{TP}{TP + FN}. \quad (3)$$

F1-score is the harmonic mean of precision and recall:

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}. \quad (4)$$

We use macro-F1 as the main metric because it gives equal importance to both classes.

5.2 Experiment Setup

The training split is used for model fitting. The validation split is used as the evaluation set because the original test file does not contain gold labels.

For TF-IDF models, we use balanced class weights. Word-level TF-IDF uses word n-grams from 1 to 2 and a maximum vocabulary size of 50,000. Character-level TF-IDF uses character n-grams from 3 to 5 and a maximum vocabulary size of 100,000.

For RuBERT-tiny2, we use the following configuration:

- model: `cointegrated/rubert-tiny2`;
- maximum sequence length: 128;
- training batch size: 8;
- evaluation batch size: 16;
- learning rate: $2 \cdot 10^{-5}$;
- number of epochs: 1;
- training subset size: 10,000 examples.

RuBERT-tiny2 is trained on a subset due to computational limitations. Classical TF-IDF models are trained on the full training set.

5.3 Baselines

We use the following baselines and competing approaches:

- Majority baseline;
- Word TF-IDF + Logistic Regression;
- Word TF-IDF + Linear SVM;
- Char TF-IDF + Logistic Regression;
- Char TF-IDF + Linear SVM;
- Word+Char TF-IDF + Logistic Regression;
- RuBERT-tiny2.

This set of models allows us to compare simple majority prediction, lexical models, character-level models, combined lexical and character-level features, and a transformer-based model.

6 Results

The results are shown in Table 3. The best model is Word+Char TF-IDF + Logistic Regression with macro-F1 of 0.7036.

Model	Accuracy	Precision	Recall	Macro F1
Word+Char TF-IDF + Logistic Regression	0.7036	0.7036	0.7036	0.7036
Char TF-IDF + Logistic Regression	0.6976	0.6976	0.6976	0.6976
Char TF-IDF + Linear SVM	0.6923	0.6923	0.6923	0.6923
Word TF-IDF + Logistic Regression	0.6723	0.6724	0.6723	0.6723
Word TF-IDF + Linear SVM	0.6634	0.6634	0.6634	0.6634
RuBERT-tiny2	0.6294	0.6298	0.6294	0.6291
Majority baseline	0.5000	0.2500	0.5000	0.3333

Table 3: Model comparison on the evaluation set.

Figure 3 presents the same comparison visually.

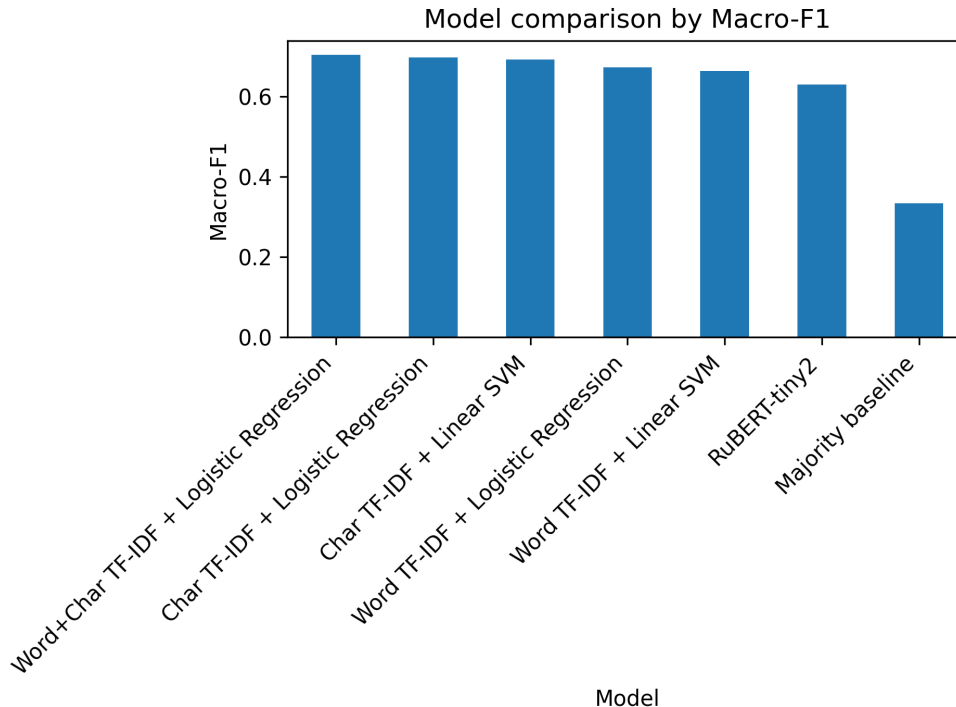


Figure 3: Model comparison by macro-F1.

The results show that character-level TF-IDF features are more effective than word-level TF-IDF features for this task. Word TF-IDF + Logistic Regression achieves macro-F1 of 0.6723, while Char TF-IDF + Logistic Regression achieves macro-F1 of 0.6976. The combined Word+Char TF-IDF representation performs best, with macro-F1 of 0.7036.

This suggests that generated text detection benefits from both lexical and surface-level stylistic information. Character n-grams may capture punctuation patterns, word endings, orthographic regularities, and other surface-level cues that are useful for distinguishing generated and human-written texts.

RuBERT-tiny2 achieves macro-F1 of 0.6291, which is lower than the best TF-IDF models. This does not necessarily mean that transformer models are unsuitable for the task. The RuBERT-tiny2 model was trained only on a 10,000-example subset for one epoch due to computational limitations. Previous

work on RuATD shows that stronger pretrained model ensembles can achieve substantially higher results [Maloyan et al., 2022].

6.1 Error Analysis

We analyze the errors of the best-performing model, Word+Char TF-IDF + Logistic Regression. The model makes 6,376 errors on the evaluation set: 3,213 false positives and 3,163 false negatives.

Error type	Count
Total errors	6,376
False positives	3,213
False negatives	3,163

Table 4: Error statistics for the best model.

False positives are human-written texts classified as machine-generated. Many false positives are written in a formal, bureaucratic, or news-like style. For example, official or administrative sentences may look formulaic and therefore resemble generated text.

False negatives are machine-generated texts classified as human-written. Many false negatives are short, fluent, or resemble historical and encyclopedic fragments. Such texts may not contain enough surface-level cues for the classifier.

Table 5 shows several representative error examples. The examples are transliterated to avoid LaTeX table encoding issues.

Text fragment	Gold	Pred.	Possible reason
V svyazi s etim v podprogrammu vklyucheny sle- duyushchie meropriyatiya...	0	1	Formal bureaucratic style
Migrantov budut obuchat russskomu yazyku pryamo na stroykakh.	0	1	Short news-like sentence
V nastoyashchee vremya zloumyshlennika razyski- vaet politsiya.	0	1	Formulaic news style
V 1881 godu pochti ne ostalos sleda uzhe ot zavoda.	1	0	Historical style
Prisutstvovali Likhaychev, Petrucci i Mazurkevich...	1	0	Named entities and human- like details

Table 5: Representative error examples for the best model. Gold label 0 means human-written text; label 1 means machine-generated text.

The error distribution is balanced between false positives and false negatives. This suggests that the model does not simply overpredict one class. Instead, the main difficulty comes from stylistic overlap between formal human-written texts and fluent machine-generated texts.

7 Conclusion

In this project, we studied machine-generated Russian text detection using the RuATD 2022 dataset. We implemented a reproducible pipeline and compared majority, word-level TF-IDF, character-level TF-IDF, combined word-and-character TF-IDF, and RuBERT-tiny2 models.

The best result was achieved by Word+Char TF-IDF + Logistic Regression, with macro-F1 of 0.7036. Character-level features improved over word-level features, which suggests that surface-level stylistic and orthographic patterns are important for detecting machine-generated Russian texts.

RuBERT-tiny2 achieved lower performance than the best TF-IDF models in our experiments. This result is likely caused by the limited fine-tuning setup: the model was trained only on a 10,000-example subset for

one epoch. Future work may include training larger transformer models, using the full training set for fine-tuning, adding stylometric features, applying ensemble methods, and evaluating models on texts generated by newer large language models.

References

- [Devlin et al., 2019] Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. (2019). Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT*.
- [Maloyan et al., 2022] Maloyan, N., Nutfullin, B., and Ilyushin, E. (2022). Dialog-22 ruatd generated text detection. *arXiv preprint arXiv:2206.08029*.
- [Shamardina et al., 2022] Shamardina, T., Mikhailov, V., Chernianskii, D., Fenogenova, A., Saidov, M., Valeeva, A., Shavrina, T., Smurov, I., Tutubalina, E., and Artemova, E. (2022). Findings of the the ruatd shared task 2022 on artificial text detection in russian. *arXiv preprint arXiv:2206.01583*.